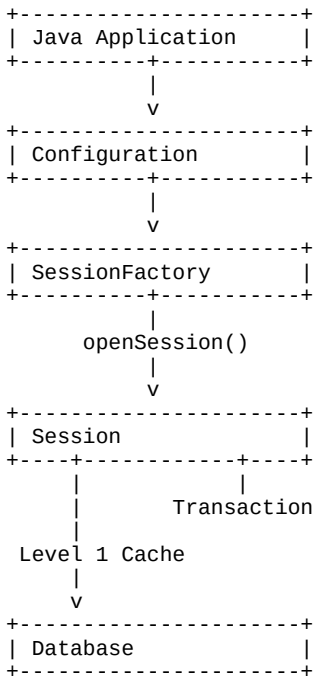
Chapter 2 - Hibernate Architecture (Detailed)

1. Introduction

Hibernate architecture defines how different components collaborate to persist Java objects into a relational database. The main components are Configuration, SessionFactory, Session, Transaction and Database.

2. Overall Architecture

Request flow from the application to the database:



3. Configuration

The Configuration object reads hibernate.cfg.xml, database properties, and entity mappings. After loading the configuration, it is used to build a SessionFactory.

4. SessionFactory

SessionFactory is a heavyweight, thread-safe object. It is usually created once during application startup and reused throughout the application. Creating it repeatedly is expensive.

5. Session

Session is lightweight and not thread-safe. It represents the persistence context, manages entities, contains the Level 1 Cache, and performs CRUD operations.

6. Transaction

Every insert, update, or delete should be executed inside a transaction. Common methods are beginTransaction(), commit(), and rollback().

7. Request Processing Flow

Typical flow in a Spring MVC application:

```
Browser
   |
Controller
   |
Service
   |
DAO / Repository
   |
Hibernate Session
   |
Database
```

8. Lifecycle

Application starts -> Configuration created -> SessionFactory created -> Session opened -> Transaction started -> CRUD operations -> Commit/Rollback -> Session closed -> SessionFactory closed when application shuts down.

9. Best Practices

• Create only one SessionFactory per database.
• Open a new Session for each request or unit of work.
• Always close Sessions.
• Keep transactions short.
• Do not share a Session between threads.

10. Interview Questions

1. Why is SessionFactory heavyweight?
2. Why is Session not thread-safe?
3. What is the difference between Session and SessionFactory?
4. Where is Level 1 Cache stored?
5. Why should every database modification use a transaction?